

Guía completa de comandos ADB Android

Edison Achalma

Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Tabla de contenidos

Introduction	5
1 ADB	5
1.1 ¿Qué es Android Debug Bridge?	5
1.2 Componentes de ADB	5
1.3 Funcionamiento	5
1.4 Ventajas de ADB	6
2 Arquitectura de ADB	6
2.1 Proceso de Conexión	6
2.2 Puertos y Conexiones	7
3 Instalación en Arch Linux	7
3.1 Método 1: Repositorios Oficiales (Recomendado)	7
3.2 Método 2: Android SDK Completo	7
3.3 Método 3: Descarga Manual	7
4 Configuración Inicial	8
4.1 1. Verificar Instalación	8
4.2 2. Configurar Permisos de Usuario	8
4.3 3. Configurar Reglas udev	9
4.4 4. Habilitar Depuración USB en Android	10
4.5 5. Primera Conexión	10

Edison Achalma  <https://orcid.org/0000-0001-6996-3364>

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; <https://credit.niso.org/>) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

5	Conexión de Dispositivos	11
5.1	Conexión USB	11
5.2	Conexión WiFi (Android 11+)	11
5.2.1	Método 1: Vinculación con Código	11
5.2.2	Método 2: Desde USB a WiFi (Android 10 y anteriores)	12
5.3	Reconexión Automática	12
5.4	Variable de Entorno ANDROID_SERIAL	13
6	Comandos Básicos	13
6.1	Gestión del Servidor ADB	13
6.2	Información del Dispositivo	13
6.3	Reiniciar Dispositivo	14
6.4	Logs y Debugging	14
7	Gestión de Aplicaciones	15
7.1	Instalar APK	15
7.2	Desinstalar Aplicación	15
7.3	Listar Aplicaciones	16
7.4	Información de Aplicación	16
7.5	Iniciar Aplicación	16
7.6	Detener Aplicación	17
7.7	Limpiar Datos de Aplicación	17
7.8	Habilitar/Deshabilitar Aplicación	17
8	Transferencia de Archivos	17
8.1	Copiar Archivos al Dispositivo (Push)	17
8.2	Copiar Archivos del Dispositivo (Pull)	18
8.3	Explorar Sistema de Archivos	18
8.4	Crear/Eliminar Archivos y Directorios	18
9	Shell de Android	19
9.1	Acceder al Shell	19
9.2	Comandos Unix Disponibles	19
9.3	Propiedades del Sistema	20
9.4	Configuraciones del Sistema	20
10	Administrador de Actividades	20
10.1	Sintaxis Básica	21
10.2	Iniciar Actividad	21
10.3	Iniciar Servicio	21
10.4	Enviar Broadcast	21
10.5	Forzar Detención	21
10.6	Matar Proceso	22
10.7	Modificar Pantalla	22
10.8	Ejemplos de Intents	22

11 Administrador de Paquetes	23
11.1 Listar Paquetes	23
11.2 Información del Paquete	23
11.3 Otorgar/Revocar Permisos	23
11.4 Habilitar/Deshabilitar Paquete	24
11.5 Limpiar Datos	24
11.6 Usuarios	24
12 Captura de Pantalla y Grabación	24
12.1 Captura de Pantalla	24
12.2 Grabación de Video	25
13 Redirección de Puertos	26
13.1 Forward (Host → Dispositivo)	26
13.2 Reverse (Dispositivo → Host)	26
14 Comandos ADB para Conectar y Instalar en Windows	26
14.1 Versión de Microsoft Windows: 10.0.22000.282	26
14.2 Cambio de directorio a la ubicación de ADB	27
14.3 Conexión con ADB	27
14.4 Instalación de Instagram APK	27
14.5 Descarga de ADB	27
14.6 Configuración	27
14.7 Comandos para conexión e instalación de aplicaciones	27
14.8 Ejemplo de salida esperada	28
15 Windows 11: Cómo descargar APK usando el Subsistema de Windows para Android y ADB	28
15.1 Paso 1: Habilitar el modo de desarrollador en el subsistema de Windows	28
15.2 Paso 2: Instalar las herramientas de la plataforma SDK	29
15.3 Paso 3: Instalar la aplicación de Android	29
16 Casos de Uso Profesionales	30
16.1 Caso 1: Instalación Masiva de APKs	30
16.2 Caso 2: Backup Automático de Aplicación	30
16.3 Caso 3: Testing Automatizado	31
16.4 Caso 4: Análisis de Rendimiento	31
16.5 Caso 5: Captura Automatizada de Screenshots	32
16.6 Caso 6: Monitoreo de Logs en Tiempo Real	33
16.7 Caso 7: Instalación y Testing Rápido	33
17 Troubleshooting	34
17.1 Problema 1: “adb: command not found”	34
17.2 Problema 2: “no permissions (missing udev rules?)”	34
17.3 Problema 3: “device offline” o “unauthorized”	35
17.4 Problema 4: “more than one device/emulator”	35
17.5 Problema 5: Conexión WiFi se desconecta	35
17.6 Problema 6: “daemon not running; starting now”	36
17.7 Problema 7: “insufficient permissions for device”	36

17.8 Problema 8: “error: protocol fault (no status)”	36
18 Automatización con Scripts	37
18.1 Script Fish: Instalador de APK Mejorado	37
18.2 Script Bash: Backup Completo	38
18.3 Alias Útiles para Fish	40
19 Referencia Rápida	41
19.1 Comandos Esenciales	41
19.2 Atajos de Teclado en Terminal	42
19.3 Variables de Entorno Útiles	43
19.4 Locations Importantes	43
19.5 Intents Comunes	43
20 Recursos Adicionales	43
20.1 Documentación Oficial	43
20.2 Comunidad	44
20.3 Herramientas Complementarias	44
21 Mejores Prácticas	44
21.1 Seguridad	44
21.2 Rendimiento	44
21.3 Desarrollo	45
22 Publicaciones Similares	45

Guía completa de comandos ADB Android

1 ADB

1.1 ¿Qué es Android Debug Bridge?

Android Debug Bridge (ADB) es una herramienta de línea de comandos versátil que permite comunicarte con dispositivos Android. Es parte esencial del SDK de Android y proporciona acceso completo al sistema operativo del dispositivo.

1.2 Componentes de ADB

ADB es un programa cliente-servidor con tres componentes:

1. Cliente

- Se ejecuta en tu computadora (Arch Linux)
- Envía comandos al servidor
- Se invoca desde la terminal con adb

2. Daemon (adbd)

- Se ejecuta en el dispositivo Android
- Ejecuta los comandos recibidos
- Proceso en segundo plano

3. Servidor

- Se ejecuta en tu computadora
- Gestiona la comunicación entre cliente y daemon
- Puerto TCP 5037
- Proceso en segundo plano

1.3 Funcionamiento

ARCH LINUX (Host)

Cliente
adb

Servidor
(puerto
5037)

USB / WiFi

DISPOSITIVO ANDROID

Daemon
(adbd)

1.4 Ventajas de ADB

Para desarrollo:

- Instalar y depurar aplicaciones
- Acceder al shell Unix del dispositivo
- Copiar archivos bidireccional
- Ejecutar comandos del sistema

Para testing:

- Automatizar pruebas
- Capturar logs y errores
- Simular eventos del usuario
- Modificar configuraciones

Para gestión:

- Instalar APKs sin Google Play
- Hacer backups de datos
- Capturar pantalla y video
- Analizar rendimiento

2 Arquitectura de ADB

2.1 Proceso de Conexión

1. Inicio del Cliente

```
adb devices
```

2. Verificación del Servidor

- Si no hay servidor corriendo → lo inicia
- El servidor se vincula al puerto 5037
- Escucha comandos de clientes

3. Detección de Dispositivos

- Escanea puertos USB
- Escanea puertos de red (5555-5585)
- Establece conexión con daemon (adbd)

4. Comunicación

- Cliente envía comando al servidor
- Servidor envía comando al daemon
- Daemon ejecuta y responde
- Respuesta llega al cliente

2.2 Puertos y Conexiones

Emuladores:

Emulador 1: consola 5554, adb 5555
Emulador 2: consola 5556, adb 5557
Emulador 3: consola 5558, adb 5559
...hasta 16 emuladores

Dispositivos Físicos:

- USB: Detección automática por vendor ID
- WiFi: Puerto 5555 (configurable)

3 Instalación en Arch Linux

3.1 Método 1: Repositorios Oficiales (Recomendado)

```
# ADB está en el paquete android-tools
sudo pacman -S android-tools
```

Verificar instalación:

```
adb version
# Salida esperada:
# Android Debug Bridge version 1.0.41
# Version 35.0.1-11580240
```

3.2 Método 2: Android SDK Completo

Si necesitas todas las herramientas de Android:

```
# 1. Instalar dependencias
sudo pacman -S jdk17-openjdk

# 2. Instalar Android SDK desde AUR
yay -S android-sdk android-sdk-platform-tools

# 3. Configurar variables de entorno
# Agregar a ~/.config/fish/config.fish
set -gx ANDROID_HOME /opt/android-sdk
set -gx PATH $PATH $ANDROID_HOME/platform-tools $ANDROID_HOME/tools
```

Recargar configuración:

```
source ~/.config/fish/config.fish
```

3.3 Método 3: Descarga Manual

```
# 1. Crear directorio
mkdir -p ~/android/platform-tools
cd ~/android/platform-tools

# 2. Descargar herramientas
curl -LO https://dl.google.com/android/repository/platform-tools-latest-linux.zip

# 3. Extraer
unzip platform-tools-latest-linux.zip

# 4. Agregar al PATH (Fish)
set -gx PATH $PATH ~/android/platform-tools

# Para hacerlo permanente:
echo 'set -gx PATH $PATH ~/android/platform-tools' >> ~/.config/fish/config.fish
```

4 Configuración Inicial

4.1 1. Verificar Instalación

```
# Versión de ADB
adb version

# Ayuda general
adb --help

# Lista de comandos
adb help
```

4.2 2. Configurar Permisos de Usuario

En Arch Linux, necesitas estar en el grupo adbusers:

```
# Verificar si el grupo existe
getent group adbusers

# Si no existe, crearlo
sudo groupadd adbusers

# Agregar tu usuario al grupo
sudo usermod -aG adbusers $USER

# Verificar
groups $USER | grep adbusers
```

Aplicar cambios (una de estas opciones):

```
# Opción 1: Recargar grupos (temporal)
newgrp adbusers

# Opción 2: Cerrar sesión y volver a iniciar

# Opción 3: Reiniciar sistema
```

4.3 3. Configurar Reglas udev

Las reglas udev permiten que ADB detecte dispositivos sin sudo.

```
# Crear archivo de reglas
sudo nano /etc/udev/rules.d/51-android.rules
```

Agregar reglas para fabricantes comunes:

```
# Google
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", MODE="0666", GROUP="adbusers"

# Samsung
SUBSYSTEM=="usb", ATTR{idVendor}=="04e8", MODE="0666", GROUP="adbusers"

# Xiaomi
SUBSYSTEM=="usb", ATTR{idVendor}=="2717", MODE="0666", GROUP="adbusers"

# Huawei
SUBSYSTEM=="usb", ATTR{idVendor}=="12d1", MODE="0666", GROUP="adbusers"

# OnePlus
SUBSYSTEM=="usb", ATTR{idVendor}=="2a70", MODE="0666", GROUP="adbusers"

# Motorola
SUBSYSTEM=="usb", ATTR{idVendor}=="22b8", MODE="0666", GROUP="adbusers"

# LG
SUBSYSTEM=="usb", ATTR{idVendor}=="1004", MODE="0666", GROUP="adbusers"

# HTC
SUBSYSTEM=="usb", ATTR{idVendor}=="0bb4", MODE="0666", GROUP="adbusers"

# Sony
SUBSYSTEM=="usb", ATTR{idVendor}=="0fce", MODE="0666", GROUP="adbusers"

# ASUS
SUBSYSTEM=="usb", ATTR{idVendor}=="0b05", MODE="0666", GROUP="adbusers"

# Oppo
SUBSYSTEM=="usb", ATTR{idVendor}=="22d9", MODE="0666", GROUP="adbusers"
```

```
# Realme
SUBSYSTEM=="usb", ATTR{idVendor}=="22d9", MODE="0666", GROUP="adbusers"
```

```
# Vivo
SUBSYSTEM=="usb", ATTR{idVendor}=="2d95", MODE="0666", GROUP="adbusers"
```

Recargar reglas:

```
sudo udevadm control --reload-rules
sudo udevadm trigger
```

4.4 4. Habilitar Depuración USB en Android

En el dispositivo Android:

1. Activar Opciones de Desarrollador:

- Ir a: Ajustes → Acerca del teléfono
- Tocar 7 veces en Número de compilación

2. Habilitar Depuración USB:

- Ir a: Ajustes → Opciones de desarrollador
- Activar: Depuración USB

3. Dispositivos Xiaomi (adicional):

- Activar: Depuración USB (Configuración de seguridad)
- Reiniciar dispositivo

4.5 5. Primera Conexión

```
# Conectar dispositivo por USB

# Iniciar servidor ADB
adb start-server

# Listar dispositivos
adb devices
```

En el dispositivo Android aparecerá un diálogo:

- “¿Permitir depuración USB?”
- Marcar: “Permitir siempre desde esta computadora”
- Aceptar

Verificar conexión:

```
adb devices
# Salida esperada:
# List of devices attached
# ABC123456789    device
```

5 Conexión de Dispositivos

5.1 Conexión USB

Básica (un solo dispositivo):

```
# Listar dispositivos
adb devices

# Salida:
# List of devices attached
# ABC123456789    device
```

Múltiples dispositivos:

```
# Listar con detalles
adb devices -l

# Salida:
# List of devices attached
# ABC123    device usb:1-1 product:phone model:Pixel_6 device:pixel6
# DEF456    device usb:1-2 product:tablet model:Tab_S8 device:tabs8
```

Especificar dispositivo:

```
# Por número de serie
adb -s ABC123456789 shell

# Solo USB (si hay uno solo)
adb -d shell

# Solo emulador (si hay uno solo)
adb -e shell
```

5.2 Conexión WiFi (Android 11+)

5.2.1 Método 1: Vinculación con Código

En el dispositivo:

1. Ir a: Ajustes → Opciones de desarrollador
2. Activar: Depuración inalámbrica
3. Tocar: Vincular dispositivo con código de vinculación
4. Aparecerá: IP, Puerto y Código de 6 dígitos

En Arch Linux:

```
# Vincular (usar IP y puerto del dispositivo)
adb pair 192.168.1.100:37859

# Ingresar código de 6 dígitos cuando se solicite
# Enter pairing code: 123456

# Conectar (usar IP mostrada en "Depuración inalámbrica")
adb connect 192.168.1.100:5555

# Verificar
adb devices
```

5.2.2 Método 2: Desde USB a WiFi (Android 10 y anteriores)

```
# 1. Conectar dispositivo por USB
adb devices

# 2. Obtener IP del dispositivo
adb shell ip addr show wlan0 | grep inet
# 0 desde el dispositivo: Ajustes → Acerca del teléfono → Estado

# 3. Habilitar TCP/IP en puerto 5555
adb tcpip 5555

# 4. Desconectar cable USB

# 5. Conectar por WiFi
adb connect 192.168.1.100:5555

# 6. Verificar
adb devices
```

Volver a USB:

```
adb usb
```

5.3 Reconexión Automática**Si se pierde conexión WiFi:**

```
# Reintentar conexión
adb connect 192.168.1.100:5555

# Si no funciona, reiniciar servidor
adb kill-server
adb start-server
adb connect 192.168.1.100:5555
```

5.4 Variable de Entorno ANDROID_SERIAL

```
# Fish
set -gx ANDROID_SERIAL ABC123456789
adb shell # Se conecta automáticamente a ABC123456789

# Bash/Zsh
export ANDROID_SERIAL=ABC123456789
adb shell

# Hacer permanente en Fish
echo 'set -gx ANDROID_SERIAL ABC123456789' >> ~/.config/fish/config.fish
```

6 Comandos Básicos

6.1 Gestión del Servidor ADB

```
# Iniciar servidor
adb start-server

# Detener servidor
adb kill-server

# Reiniciar servidor
adb kill-server && adb start-server

# Verificar versión
adb version

# Ver puerto del servidor
# (siempre es 5037 por defecto)
```

6.2 Información del Dispositivo

```
# Listar dispositivos conectados
adb devices
adb devices -l # con detalles

# Shell del dispositivo
adb shell

# Ejecutar comando único
adb shell ls /sdcard

# Obtener información del sistema
```

```
adb shell getprop

# Versión de Android
adb shell getprop ro.build.version.release

# Modelo del dispositivo
adb shell getprop ro.product.model

# Fabricante
adb shell getprop ro.product.manufacturer

# Número de serie
adb get-serialno

# Estado del dispositivo
adb get-state
# Posibles valores: device, offline, bootloader, recovery
```

6.3 Reiniciar Dispositivo

```
# Reinicio normal
adb reboot

# Reiniciar en recovery
adb reboot recovery

# Reiniciar en bootloader
adb reboot bootloader

# Reiniciar en modo fastboot
adb reboot-bootloader
```

6.4 Logs y Debugging

```
# Ver logs en tiempo real
adb logcat

# Filtrar por tag
adb logcat ActivityManager:I *:S

# Guardar logs en archivo
adb logcat > logs.txt

# Limpiar buffer de logs
adb logcat -c
```

```
# Ver logs de kernel
adb shell dmesg

# Obtener bug report
adb bugreport bugreport.zip
```

7 Gestión de Aplicaciones

7.1 Instalar APK

```
# Instalación básica
adb install app.apk

# Reinstalar manteniendo datos
adb install -r app.apk

# Instalar en almacenamiento externo
adb install -s app.apk

# Permitir downgrade
adb install -d app.apk

# Instalar APK de prueba
adb install -t app.apk

# Otorgar todos los permisos
adb install -g app.apk

# Instalar en dispositivo específico
adb -s ABC123 install app.apk

# Instalar múltiples APKs (split APKs)
adb install-multiple base.apk config.apk

# Instalación incremental (Android 11+)
adb install --incremental app.apk
```

7.2 Desinstalar Aplicación

```
# Desinstalar
adb uninstall com.example.app

# Desinstalar pero mantener datos
adb uninstall -k com.example.app
```

```
# Desinstalar del dispositivo específico
adb -s ABC123 uninstall com.example.app
```

7.3 Listar Aplicaciones

```
# Todas las aplicaciones
adb shell pm list packages

# Solo aplicaciones de terceros
adb shell pm list packages -3

# Solo aplicaciones del sistema
adb shell pm list packages -s

# Aplicaciones habilitadas
adb shell pm list packages -e

# Aplicaciones deshabilitadas
adb shell pm list packages -d

# Buscar por nombre
adb shell pm list packages | grep chrome

# Con ruta del APK
adb shell pm list packages -f
```

7.4 Información de Aplicación

```
# Ruta del APK
adb shell pm path com.android.chrome

# Información completa
adb shell dumpsys package com.android.chrome

# Permisos
adb shell dumpsys package com.android.chrome | grep permission

# Versión
adb shell dumpsys package com.android.chrome | grep versionName
```

7.5 Iniciar Aplicación

```
# Iniciar actividad principal
adb shell monkey -p com.android.chrome -c android.intent.category.LAUNCHER 1
```

```
# Iniciar actividad específica
adb shell am start -n com.android.chrome/com.google.android.apps.chrome.Main

# Iniciar con intent
adb shell am start -a android.intent.action.VIEW -d https://google.com
```

7.6 Detener Aplicación

```
# Forzar detención
adb shell am force-stop com.android.chrome

# Matar proceso
adb shell am kill com.android.chrome
```

7.7 Limpiar Datos de Aplicación

```
# Borrar todos los datos y caché
adb shell pm clear com.android.chrome
```

7.8 Habilitar/Deshabilitar Aplicación

```
# Deshabilitar
adb shell pm disable-user com.example.app

# Habilitar
adb shell pm enable com.example.app
```

8 Transferencia de Archivos

8.1 Copiar Archivos al Dispositivo (Push)

```
# Sintaxis básica
adb push archivo_local ruta_dispositivo

# Ejemplos:
adb push documento.pdf /sdcard/Download/
adb push foto.jpg /sdcard/DCIM/
adb push video.mp4 /sdcard/Movies/
adb push musica.mp3 /sdcard/Music/

# Copiar directorio completo
adb push carpeta/ /sdcard/carpeta/

# Con dispositivo específico
adb -s ABC123 push archivo.txt /sdcard/
```

8.2 Copiar Archivos del Dispositivo (Pull)

```
# Sintaxis básica
adb pull ruta_dispositivo archivo_local

# Ejemplos:
adb pull /sdcard/Download/documento.pdf ./
adb pull /sdcard/DCIM/Camera/foto.jpg ~/Imágenes/
adb pull /sdcard/WhatsApp/Media/ ./whatsapp_backup/

# Directorio completo
adb pull /sdcard/Download/ ./descargas/

# Con dispositivo específico
adb -s ABC123 pull /sdcard/foto.jpg ./
```

8.3 Explorar Sistema de Archivos

```
# Listar contenido de directorio
adb shell ls /sdcard/

# Listar con detalles
adb shell ls -la /sdcard/

# Árbol de directorios (si está disponible)
adb shell find /sdcard/ -type d

# Buscar archivos
adb shell find /sdcard/ -name "*.pdf"

# Ver contenido de archivo
adb shell cat /sdcard/archivo.txt

# Espacio disponible
adb shell df -h
```

8.4 Crear/Eliminar Archivos y Directorios

```
# Crear directorio
adb shell mkdir /sdcard/MiCarpeta

# Crear archivo
adb shell touch /sdcard/archivo.txt

# Escribir en archivo
adb shell "echo 'Hola' > /sdcard/archivo.txt"
```

```
# Eliminar archivo
adb shell rm /sdcard/archivo.txt

# Eliminar directorio vacío
adb shell rmdir /sdcard/MiCarpeta

# Eliminar directorio con contenido
adb shell rm -rf /sdcard/MiCarpeta
```

9 Shell de Android

9.1 Acceder al Shell

```
# Shell interactivo
adb shell

# Comando único
adb shell ls /system/bin

# Múltiples comandos
adb shell "cd /sdcard && ls -l"

# Como root (si disponible)
adb root
adb shell

# Ahora estarás como root (#)
```

9.2 Comandos Unix Disponibles

```
# Listar herramientas disponibles
adb shell ls /system/bin

# Ayuda de herramientas
adb shell toybox --help
adb shell ls --help
```

Comandos comunes:

- ls - Listar archivos
- cd - Cambiar directorio
- pwd - Mostrar directorio actual
- cat - Ver contenido de archivo
- grep - Buscar texto
- find - Buscar archivos
- ps - Listar procesos

- top - Monitor de procesos
- kill - Terminar proceso
- df - Espacio en disco
- du - Uso de disco
- cp - Copiar
- mv - Mover
- rm - Eliminar
- chmod - Cambiar permisos
- chown - Cambiar propietario

9.3 Propiedades del Sistema

```
# Listar todas las propiedades
adb shell getprop

# Ver propiedad específica
adb shell getprop ro.build.version.sdk

# Establecer propiedad (requiere root)
adb shell setprop debug.myapp.level 5

# Propiedades útiles:
adb shell getprop ro.product.model           # Modelo
adb shell getprop ro.build.version.release   # Versión Android
adb shell getprop ro.build.version.sdk       # Nivel API
adb shell getprop ro.product.manufacturer    # Fabricante
adb shell getprop ro.serialno                 # Serial
adb shell getprop net.hostname               # Hostname
```

9.4 Configuraciones del Sistema

```
# Ver configuración
adb shell settings get global wifi_on
adb shell settings get system screen_brightness

# Establecer configuración
adb shell settings put global airplane_mode_on 0
adb shell settings put system screen_brightness 100

# Listar todas
adb shell settings list global
adb shell settings list system
adb shell settings list secure
```

10 Administrador de Actividades

El Administrador de Actividades (am) controla actividades, servicios e intents.

10.1 Sintaxis Básica

```
# Desde shell
adb shell
am comando

# Comando directo
adb shell am comando
```

10.2 Iniciar Actividad

```
# Iniciar actividad por acción
adb shell am start -a android.intent.action.VIEW

# Iniciar actividad por componente
adb shell am start -n com.android.chrome/com.google.android.apps.chrome.Main

# Abrir URL
adb shell am start -a android.intent.action.VIEW -d https://google.com

# Abrir archivo
adb shell am start -a android.intent.action.VIEW -d file:///sdcard/documento.pdf

# Con espera hasta completar
adb shell am start -W -n com.android.chrome/com.google.android.apps.chrome.Main

# Habilitar debugging
adb shell am start -D -n com.example.app/.MainActivity
```

10.3 Iniciar Servicio

```
adb shell am startservice -n com.example.app/.MyService
```

10.4 Enviar Broadcast

```
# Broadcast simple
adb shell am broadcast -a android.intent.action.BOOT_COMPLETED

# Con datos extra
adb shell am broadcast -a com.example.ACTION --es key value
```

10.5 Forzar Detención

```
adb shell am force-stop com.android.chrome
```

10.6 Matar Proceso

```
# Matar app
adb shell am kill com.android.chrome

# Matar todos los procesos en segundo plano
adb shell am kill-all
```

10.7 Modificar Pantalla

```
# Cambiar tamaño de pantalla
adb shell am display-size 1920x1080

# Restablecer
adb shell am display-size reset

# Cambiar densidad (DPI)
adb shell am display-density 480

# Restablecer
adb shell am display-density reset
```

10.8 Ejemplos de Intents

```
# Abrir configuración WiFi
adb shell am start -a android.settings.WIFI_SETTINGS

# Abrir ajustes del sistema
adb shell am start -a android.settings.SETTINGS

# Abrir configuración de aplicación específica
adb shell am start -a android.settings.APPLICATION_DETAILS_SETTINGS -d package:com.ex

# Hacer llamada
adb shell am start -a android.intent.action.CALL -d tel:123456789

# Enviar SMS
adb shell am start -a android.intent.action.SENDTO -d sms:123456789 --es sms_body "Ho

# Abrir cámara
adb shell am start -a android.media.action.IMAGE_CAPTURE

# Compartir texto
adb shell am start -a android.intent.action.SEND --es android.intent.extra.TEXT "Text
```

11 Administrador de Paquetes

El Administrador de Paquetes (pm) gestiona aplicaciones instaladas.

11.1 Listar Paquetes

```
# Todos los paquetes
adb shell pm list packages

# Solo terceros
adb shell pm list packages -3

# Solo sistema
adb shell pm list packages -s

# Habilitados
adb shell pm list packages -e

# Deshabilitados
adb shell pm list packages -d

# Con archivo APK
adb shell pm list packages -f

# Buscar específico
adb shell pm list packages | grep chrome
```

11.2 Información del Paquete

```
# Ruta del APK
adb shell pm path com.android.chrome

# Información completa
adb shell dumpsys package com.android.chrome

# Permisos
adb shell pm list permissions

# Permisos de paquete
adb shell dumpsys package com.android.chrome | grep permission

# Grupos de permisos
adb shell pm list permission-groups
```

11.3 Otorgar/Revocar Permisos

```
# Otorgar permiso
adb shell pm grant com.example.app android.permission.CAMERA

# Revocar permiso
adb shell pm revoke com.example.app android.permission.CAMERA

# Permisos comunes:
# android.permission.CAMERA
# android.permission.READ_EXTERNAL_STORAGE
# android.permission.WRITE_EXTERNAL_STORAGE
# android.permission.ACCESS_FINE_LOCATION
# android.permission.RECORD_AUDIO
```

11.4 Habilitar/Deshabilitar Paquete

```
# Deshabilitar
adb shell pm disable-user com.example.app

# Habilitar
adb shell pm enable com.example.app
```

11.5 Limpiar Datos

```
# Borrar datos de aplicación
adb shell pm clear com.android.chrome
```

11.6 Usuarios

```
# Listar usuarios
adb shell pm list users

# Crear usuario
adb shell pm create-user "Nombre Usuario"

# Eliminar usuario
adb shell pm remove-user USER_ID

# Máximo de usuarios
adb shell pm get-max-users
```

12 Captura de Pantalla y Grabación

12.1 Captura de Pantalla

```
# Capturar y guardar en dispositivo
adb shell screencap /sdcard/screenshot.png

# Capturar y descargar
adb shell screencap /sdcard/screenshot.png
adb pull /sdcard/screenshot.png ./
adb shell rm /sdcard/screenshot.png

# Captura directa (un solo comando)
adb exec-out screencap -p > screenshot.png

# Con timestamp
adb exec-out screencap -p > screenshot_$(date +%Y%m%d_%H%M%S).png
```

12.2 Grabación de Video

```
# Grabar video (hasta 3 minutos por defecto)
adb shell screenrecord /sdcard/video.mp4

# Detener con Ctrl+C

# Descargar
adb pull /sdcard/video.mp4 ./

# Con límite de tiempo (segundos)
adb shell screenrecord --time-limit 30 /sdcard/video.mp4

# Especificar tamaño
adb shell screenrecord --size 1280x720 /sdcard/video.mp4

# Cambiar bitrate (por defecto 20 Mbps)
adb shell screenrecord --bit-rate 8000000 /sdcard/video.mp4

# Verbose (mostrar info)
adb shell screenrecord --verbose /sdcard/video.mp4

# Rotar video 90 grados
adb shell screenrecord --rotate /sdcard/video.mp4
```

Limitaciones de screenrecord:

- No graba audio
- Máximo 3 minutos por video
- No funciona en Wear OS
- No soporta rotación durante grabación

13 Redirección de Puertos

13.1 Forward (Host → Dispositivo)

```
# Redirigir puerto host 6100 a dispositivo 7100
adb forward tcp:6100 tcp:7100

# Listar redirecciones
adb forward --list

# Eliminar redirección específica
adb forward --remove tcp:6100

# Eliminar todas
adb forward --remove-all

# Ejemplo: acceder a servidor en dispositivo desde host
# Si hay servidor en dispositivo:puerto 8080
adb forward tcp:8080 tcp:8080
# Ahora accesible en: http://localhost:8080
```

13.2 Reverse (Dispositivo → Host)

```
# Redirigir puerto dispositivo 9000 a host 9000
adb reverse tcp:9000 tcp:9000

# Listar
adb reverse --list

# Eliminar
adb reverse --remove tcp:9000

# Eliminar todas
adb reverse --remove-all

# Ejemplo: dispositivo accede a servidor en host
# Si hay servidor en host:puerto 3000
adb reverse tcp:3000 tcp:3000
# Dispositivo puede acceder a http://localhost:3000
```

14 Comandos ADB para Conectar y Instalar en Windows

14.1 Versión de Microsoft Windows: 10.0.22000.282

(c) Microsoft Corporation. Todos los derechos reservados.

14.2 Cambio de directorio a la ubicación de ADB

```
C:\Users\achalmaedison>cd "C:\Users\achalmaedison\Downloads\ADB"
```

14.3 Conexión con ADB

```
C:\Users\achalmaedison\Downloads\ADB>adb connect 127.0.0.1:58526
```

Salida esperada:

```
Ya conectado a 127.0.0.1:58526
```

14.4 Instalación de Instagram APK

```
C:\Users\achalmaedison\Downloads\ADB>adb install Instagram.apk
```

Salida esperada:

```
Realizando Instalación en Streaming
```

```
Instalación Exitosa
```

Este es un ejemplo de cómo utilizar los comandos ADB en un entorno Windows.

14.5 Descarga de ADB

Puedes descargar ADB desde el siguiente enlace:

[Plataforma de herramientas de Android](#)

14.6 Configuración

1. Configurar el subsistema de Android.
2. Activar el modo desarrollador.
3. Copiar la dirección IP del dispositivo.
4. Abrir **CMD** como administrador.
5. Navegar a la carpeta platform-tools ubicada en C:\platform-tools:

```
cd C:\platform-tools
```

14.7 Comandos para conexión e instalación de aplicaciones

Conectar el dispositivo:

```
adb connect 127.0.0.1:58526
```

Instalar aplicaciones:

```
adb install gmail.apk
```

```
adb install facebook-354-0-0-22-110.apk
```

Nota: Si los comandos no funcionan, intenta reiniciar el sistema.

14.8 Ejemplo de salida esperada

```
Microsoft Windows [Version 10.0.22000.527]
```

```
(c) Microsoft Corporation. All rights reserved.
```

```
C:\platform-tools>adb connect 172.28.22.78
```

```
* daemon not running; starting now at tcp:5037
```

```
* daemon started successfully
```

```
connected to 172.28.22.78:5555
```

```
C:\platform-tools>adb install termux-app_v0.118.0+github-debug_arm64-v8a.apk
```

```
Performing Streamed Install
```

```
Success
```

```
C:\platform-tools>
```

15 Windows 11: Cómo descargar APK usando el Subsistema de Windows para Android y ADB

Aquí te explicamos cómo descargar un archivo APK para instalar una aplicación de Android en tu PC con Windows 11 usando el Subsistema de Windows para Android. Puedes instalar el Subsistema de Windows para Android manualmente en tu PC con su archivo Msixbundle siguiendo nuestra [guía aquí](#).

15.1 Paso 1: Habilitar el modo de desarrollador en el subsistema de Windows

1. Instala el [Subsistema de Windows para Android](#).
2. Abre la aplicación 'Subsistema de Windows para Android' en tu PC. Para ello, presiona la tecla **Windows** y busca **Subsistema de Windows para Android**.
3. Haz clic en la aplicación para abrirla.
4. Dentro de la aplicación, activa el **Modo Desarrollador**.

15.2 Paso 2: Instalar las herramientas de la plataforma SDK

1. Visita la página de herramientas de la plataforma SDK de Google [aquí](#).
2. Descarga **SDK Platform-Tools para Windows**.
3. Acepta los términos y condiciones y haz clic en el botón de descarga.
4. Se descargará un archivo ZIP llamado **platform-tools_rXX.X.X-windows.zip** (la versión puede variar).
5. Crea una carpeta separada en el Explorador de Windows, por ejemplo, C:\Plataforma-Tools.
6. Mueve el archivo ZIP descargado a esta carpeta.
7. Haz clic derecho en el archivo y selecciona **Extraer todo**, luego haz clic en **Extraer**.
8. Abre la carpeta platform-tools, donde encontrarás adb.exe y otros archivos.

15.3 Paso 3: Instalar la aplicación de Android

1. Abre la carpeta **platform-tools**.
2. Haz clic en la barra de direcciones, escribe `cmd` y presiona **Enter**.
3. Se abrirá una ventana de comandos en la ubicación de la carpeta **platform-tools**.
4. Descarga el archivo APK de la aplicación de Android que deseas instalar.
 - Por ejemplo, para instalar Snapchat, busca **Snapchat APK** en Google y descarga el archivo de una fuente confiable.
 - Renombra el archivo a algo simple, como `snapchat.apk`, y muévelo a la carpeta **platform-tools**.
5. Abre el **Subsistema de Windows para Android** y copia la dirección **IP** en la opción de **Modo Desarrollador**.
6. En la ventana de comandos, ejecuta el siguiente comando:

```
adb.exe connect [DIRECCIÓN_IP]
```

Ejemplo:

```
adb.exe connect 127.0.0.1:12345
```

7. Luego, instala la aplicación ejecutando:

```
adb.exe install [NOMBRE_DEL_APK]
```

Ejemplo:

```
adb.exe install snapchat.apk
```

8. Cuando la instalación finalice, verás el mensaje **Success**.
9. Cierra la ventana de comandos.
10. Abre la aplicación en tu PC escribiendo su nombre en el menú Inicio (por ejemplo, **Snapchat**).

16 Casos de Uso Profesionales

16.1 Caso 1: Instalación Masiva de APKs

Escenario: Instalar múltiples APKs en varios dispositivos.

```
#!/bin/bash
# instalar_apks.sh

# Directorio con APKs
APK_DIR="./apks"

# Obtener lista de dispositivos
devices=$(adb devices | grep -v "List" | grep "device$" | awk '{print $1}')

# Para cada dispositivo
for device in $devices; do
    echo "Instalando en dispositivo: $device"

    # Para cada APK
    for apk in "$APK_DIR"/*.apk; do
        echo "  Instalando: $(basename $apk)"
        adb -s "$device" install -r "$apk"
    done
done

echo "Instalación completada"
```

Usar:

```
chmod +x instalar_apks.sh
./instalar_apks.sh
```

16.2 Caso 2: Backup Automático de Aplicación

Escenario: Respalidar datos de aplicación específica.

```
#!/bin/bash
# backup_app.sh

PACKAGE="com.example.app"
BACKUP_DIR="./backups/$(date +%Y%m%d_%H%M%S)"

mkdir -p "$BACKUP_DIR"

echo "Respaldando $PACKAGE..."

# Backup de APK
echo "1. Respaldando APK..."
```

```
APK_PATH=$(adb shell pm path "$PACKAGE" | cut -d: -f2 | tr -d '\r')
adb pull "$APK_PATH" "$BACKUP_DIR/app.apk"

# Backup de datos (requiere app debuggable o root)
echo "2. Respalando datos..."
adb backup -f "$BACKUP_DIR/data.ab" -noapk "$PACKAGE"

# Backup de base de datos (si accesible)
echo "3. Respalando bases de datos..."
adb pull /data/data/"$PACKAGE"/databases "$BACKUP_DIR/databases/"

echo "Backup completado en: $BACKUP_DIR"
```

16.3 Caso 3: Testing Automatizado

Escenario: Ejecutar suite de tests en dispositivo.

```
#!/bin/bash
# run_tests.sh

PACKAGE="com.example.app"
TEST_PACKAGE="com.example.app.test"

echo "Iniciando tests..."

# Instalar APKs
adb install -r app.apk
adb install -r app-test.apk

# Ejecutar tests
adb shell am instrument -w \
    -e package "$PACKAGE" \
    "$TEST_PACKAGE/androidx.test.runner.AndroidJUnitRunner"

# Generar reporte
adb pull /sdcard/test-results.xml ./
```

16.4 Caso 4: Análisis de Rendimiento

Escenario: Capturar métricas de rendimiento de aplicación.

```
#!/bin/bash
# performance_test.sh

PACKAGE="com.example.app"
DURATION=60 # segundos

echo "Analizando rendimiento de $PACKAGE por $DURATION segundos..."
```

```
# Limpiar logs
adb logcat -c

# Iniciar app
adb shell monkey -p "$PACKAGE" -c android.intent.category.LAUNCHER 1

# Capturar CPU/Memoria
adb shell top -n 1 -d 1 -m 10 > cpu_usage.txt &
TOP_PID=$!

# Esperar duración
sleep "$DURATION"

# Detener captura
kill $TOP_PID

# Obtener memoria detallada
adb shell dumpsys meminfo "$PACKAGE" > memory_info.txt

# Obtener logs
adb logcat -d > app_logs.txt

echo "Análisis completado"
```

16.5 Caso 5: Captura Automatizada de Screenshots

Escenario: Capturar screenshots para documentación.

```
#!/bin/bash
# capture_screenshots.sh

OUTPUT_DIR="./screenshots/${date +%Y%m%d_%H%M%S}"
mkdir -p "$OUTPUT_DIR"

SCREENS=(
    "com.example.app/.MainActivity:Pantalla Principal"
    "com.example.app/.SettingsActivity:Configuración"
    "com.example.app/.ProfileActivity:Perfil"
)

for screen in "${SCREENS[@]}; do
    ACTIVITY=$(echo "$screen" | cut -d: -f1)
    NAME=$(echo "$screen" | cut -d: -f2)

    echo "Capturando: $NAME"

    # Abrir actividad
```

```
adb shell am start -n "$ACTIVITY"
sleep 2

# Capturar
FILENAME=$(echo "$NAME" | tr ' ' '_' | tr '[:upper:]' '[:lower:]')
adb exec-out screencap -p > "$OUTPUT_DIR/${FILENAME}.png"

sleep 1
done

echo "Screenshots guardados en: $OUTPUT_DIR"
```

16.6 Caso 6: Monitoreo de Logs en Tiempo Real

Escenario: Filtrar logs de aplicación específica.

```
#!/bin/bash
# monitor_logs.sh

PACKAGE="com.example.app"

# Obtener PID
PID=$(adb shell ps | grep "$PACKAGE" | awk '{print $2}')

echo "Monitoreando logs de $PACKAGE (PID: $PID)"
echo "Presiona Ctrl+C para detener"

# Filtrar logs por PID
adb logcat --pid="$PID" -v time
```

16.7 Caso 7: Instalación y Testing Rápido

Escenario: Ciclo desarrollo-testing rápido.

```
#!/bin/bash
# quick_test.sh

APK="app-debug.apk"
PACKAGE="com.example.app"

echo "1. Desinstalando versión anterior..."
adb uninstall "$PACKAGE" 2>/dev/null

echo "2. Instalando nueva versión..."
adb install "$APK"

echo "3. Otorgando permisos..."
adb shell pm grant "$PACKAGE" android.permission.CAMERA
```

```
adb shell pm grant "$PACKAGE" android.permission.READ_EXTERNAL_STORAGE
adb shell pm grant "$PACKAGE" android.permission.WRITE_EXTERNAL_STORAGE

echo "4. Iniciando app..."
adb shell monkey -p "$PACKAGE" -c android.intent.category.LAUNCHER 1

echo "5. Mostrando logs..."
adb logcat -c
adb logcat | grep "$PACKAGE"
```

17 Troubleshooting

17.1 Problema 1: “adb: command not found”

Causa: ADB no está instalado o no está en PATH.

Soluciones:

```
# 1. Verificar instalación
which adb

# 2. Si no está, instalar
sudo pacman -S android-tools

# 3. Verificar PATH
echo $PATH | grep android

# 4. Si instalaste manualmente, agregar a PATH
set -gx PATH $PATH ~/android/platform-tools
```

17.2 Problema 2: “no permissions (missing udev rules?)”

Causa: Reglas udev faltantes o usuario no en grupo adbusers.

Soluciones:

```
# 1. Verificar grupo
groups | grep adbusers

# 2. Agregar a grupo
sudo usermod -aG adbusers $USER
newgrp adbusers

# 3. Crear/verificar reglas udev
sudo nano /etc/udev/rules.d/51-android.rules
# Ver sección "Configuración Inicial"

# 4. Recargar reglas
sudo udevadm control --reload-rules
sudo udevadm trigger
```

```
# 5. Reconectar dispositivo
```

17.3 Problema 3: “device offline” o “unauthorized”

Causa: Dispositivo no autorizado o conexión interrumpida.

Soluciones:

```
# 1. Revocar autorizaciones en dispositivo
# Android: Opciones desarrollador → Revocar autorizaciones USB

# 2. Reconectar dispositivo

# 3. Aceptar diálogo de autorización

# 4. Reiniciar servidor adb
adb kill-server
adb start-server
adb devices
```

17.4 Problema 4: “more than one device/emulator”

Causa: Múltiples dispositivos conectados sin especificar cuál usar.

Soluciones:

```
# 1. Listar dispositivos
adb devices

# 2. Especificar por serial
adb -s ABC123456789 shell

# 3. O establecer variable de entorno
set -gx ANDROID_SERIAL ABC123456789
adb shell

# 4. Usar -d (solo USB) o -e (solo emulador)
adb -d shell
adb -e shell
```

17.5 Problema 5: Conexión WiFi se desconecta

Causa: Red inestable o dispositivo cambia de red.

Soluciones:

```
# 1. Verificar que ambos estén en misma red
# Dispositivo: Ajustes → WiFi
# Host: ip addr show

# 2. Reconectar
```

```
adb connect 192.168.1.100:5555

# 3. Si falla, reiniciar desde USB
adb usb
adb tcpip 5555
adb connect 192.168.1.100:5555

# 4. Ping para verificar conectividad
ping 192.168.1.100
```

17.6 Problema 6: “daemon not running; starting now”

Causa: Servidor ADB no estaba corriendo (normal).

No es un error. ADB inicia automáticamente el servidor.

```
# Si quieres iniciar manualmente:
adb start-server
```

17.7 Problema 7: “insufficient permissions for device”

Causa: Permisos USB incorrectos.

Soluciones:

```
# 1. Verificar reglas udev (ver Problema 2)

# 2. Verificar vendor ID del dispositivo
lsusb
# Buscar tu dispositivo Android

# 3. Agregar vendor ID a reglas udev
sudo nano /etc/udev/rules.d/51-android.rules
# SUBSYSTEM=="usb", ATTR{idVendor}=="XXXX", MODE="0666", GROUP="adbusers"

# 4. Recargar reglas
sudo udevadm control --reload-rules
```

17.8 Problema 8: “error: protocol fault (no status)”

Causa: Comunicación corrupta con dispositivo.

Soluciones:

```
# 1. Desconectar y reconectar dispositivo

# 2. Reiniciar servidor ADB
adb kill-server
adb start-server

# 3. Reiniciar dispositivo
adb reboot
```

```
# 4. Como último recurso, reiniciar computadora
```

18 Automatización con Scripts

18.1 Script Fish: Instalador de APK Mejorado

Crear ~/.config/fish/functions/adb-install-smart.fish:

```
function adb-install-smart --description "Instalar APK con opciones inteligentes"
  if test (count $argv) -lt 1
    echo "Uso: adb-install-smart <archivo.apk> [opciones]"
    echo ""
    echo "Opciones:"
    echo "  -r : Reinstalar manteniendo datos"
    echo "  -g : Otorgar todos los permisos"
    echo "  -t : APK de prueba"
    echo "  -d : Permitir downgrade"
    return 1
  end

  set apk $argv[1]
  set options

  # Parsear opciones
  for arg in $argv[2..-1]
    set options $options $arg
  end

  # Verificar que el archivo existe
  if not test -f "$apk"
    echo "Error: Archivo no encontrado: $apk"
    return 1
  end

  # Verificar extensión
  if not string match -q "*.apk" "$apk"
    echo "Error: El archivo debe ser .apk"
    return 1
  end

  # Obtener package name
  set package (aapt dump badging "$apk" 2>/dev/null | grep package: | awk '{print $2}')

  echo "Instalando: $apk"
  if test -n "$package"
    echo "Paquete: $package"
  end
end
```

```
# Instalar
adb install $options "$apk"

if test $status -eq 0
    echo "  Instalación exitosa"

    # Si se otorgaron permisos, mostrarlos
    if string match -q "*-g*" "$options"
        echo "Permisos otorgados automáticamente"
    end
else
    echo "  Error en la instalación"
    return 1
end
end
end
```

Usar:

```
adb-install-smart app.apk
adb-install-smart app.apk -r -g
```

18.2 Script Bash: Backup Completo**Crear ~/bin/adb-backup-full:**

```
#!/bin/bash
# adb-backup-full - Backup completo del dispositivo

set -e

BACKUP_DIR="$HOME/android-backups/$(date +%Y%m%d_%H%M%S)"
DEVICE_SERIAL=""

# Parsear argumentos
while getopts "s:d:" opt; do
    case $opt in
        s) DEVICE_SERIAL="-s $OPTARG" ;;
        d) BACKUP_DIR="$OPTARG" ;;
        *) echo "Uso: $0 [-s serial] [-d directorio]"; exit 1 ;;
    esac
done

mkdir -p "$BACKUP_DIR"

echo "====="
echo "Backup Completo de Dispositivo Android"
echo "====="
echo "Directorio: $BACKUP_DIR"
```

```
echo ""

# Información del dispositivo
echo "1. Capturando información del dispositivo..."
adb $DEVICE_SERIAL shell getprop > "$BACKUP_DIR/device_info.txt"
adb $DEVICE_SERIAL shell "dumpsys battery" > "$BACKUP_DIR/battery_info.txt"

# Lista de aplicaciones
echo "2. Listando aplicaciones instaladas..."
adb $DEVICE_SERIAL shell pm list packages -f > "$BACKUP_DIR/packages.txt"
adb $DEVICE_SERIAL shell pm list packages -3 > "$BACKUP_DIR/packages_user.txt"

# Backup de APKs de usuario
echo "3. Respaldando APKs de usuario..."
mkdir -p "$BACKUP_DIR/apks"
USER_APPS=$(adb $DEVICE_SERIAL shell pm list packages -3 | cut -d: -f2)
for app in $USER_APPS; do
    echo "    Respaldando: $app"
    APK_PATH=$(adb $DEVICE_SERIAL shell pm path "$app" | cut -d: -f2 | tr -d '\r')
    adb $DEVICE_SERIAL pull "$APK_PATH" "$BACKUP_DIR/apks/${app}.apk" 2>/dev/null ||
done

# Backup de datos
echo "4. Respaldando datos de usuario..."
adb $DEVICE_SERIAL backup -f "$BACKUP_DIR/data.ab" -all -apk -shared

# Contactos (si accesible)
echo "5. Respaldando contactos..."
adb $DEVICE_SERIAL pull /data/data/com.android.providers.contacts/databases "$BACKUP_

# Fotos y videos
echo "6. Respaldando multimedia..."
adb $DEVICE_SERIAL pull /sdcard/DCIM "$BACKUP_DIR/DCIM/" 2>/dev/null || true
adb $DEVICE_SERIAL pull /sdcard/Pictures "$BACKUP_DIR/Pictures/" 2>/dev/null || true
adb $DEVICE_SERIAL pull /sdcard/Download "$BACKUP_DIR/Download/" 2>/dev/null || true

# Crear archivo de metadatos
echo "7. Creando metadatos..."
cat > "$BACKUP_DIR/README.txt" << EOF
Backup de Dispositivo Android
Fecha: $(date)
Dispositivo: $(adb $DEVICE_SERIAL shell getprop ro.product.model)
Android: $(adb $DEVICE_SERIAL shell getprop ro.build.version.release)
Serial: $(adb $DEVICE_SERIAL get-serialno)

Contenido:
- device_info.txt: Propiedades del sistema
```

```

- packages.txt: Lista de todos los paquetes
- packages_user.txt: Solo paquetes de usuario
- apks/: APKs de aplicaciones de usuario
- data.ab: Backup de datos (usar 'adb restore data.ab')
- DCIM/, Pictures/, Download/: Multimedia
EOF

echo ""
echo "=====
echo "Backup completado exitosamente"
echo "Ubicación: $BACKUP_DIR"
echo "=====

```

Hacer ejecutable y usar:

```

chmod +x ~/bin/adb-backup-full
adb-backup-full
adb-backup-full -s ABC123456789
adb-backup-full -d ~/mis-backups/backup-especial

```

18.3 Alias Útiles para Fish

Agregar a ~/.config/fish/config.fish:

```

# =====
# ALIAS DE ADB
# =====

# Básicos
alias adb-devices="adb devices -l"
alias adb-restart="adb kill-server && adb start-server"
alias adb-logcat="adb logcat -v time"

# Dispositivo
alias adb-model="adb shell getprop ro.product.model"
alias adb-android="adb shell getprop ro.build.version.release"
alias adb-battery="adb shell dumpsys battery"
alias adb-wifi="adb shell svc wifi enable"
alias adb-wifi-off="adb shell svc wifi disable"

# Aplicaciones
alias adb-apps="adb shell pm list packages -3"
alias adb-apps-all="adb shell pm list packages"
alias adb-uninstall="adb uninstall"
alias adb-clear="adb shell pm clear"

# Archivos
alias adb-pull-dcim="adb pull /sdcard/DCIM/ ./"

```

```
alias adb-pull-download="adb pull /sdcard/Download/ ./"
alias adb-ls="adb shell ls -la"

# Captura
alias adb-screenshot="adb exec-out screencap -p > screenshot_(date +%Y%m%d_%H%M%S).png"
alias adb-record="adb shell screenrecord /sdcard/video.mp4"

# Desarrollo
alias adb-install-debug="adb install -r -t -g"
alias adb-logs="adb logcat -v time | grep -i"

# Conexión
alias adb-wifi-connect="adb tcpip 5555"
alias adb-usb="adb usb"

# Reinicio
alias adb-reboot="adb reboot"
alias adb-recovery="adb reboot recovery"
alias adb-bootloader="adb reboot bootloader"
```

Recargar:

```
source ~/.config/fish/config.fish
```

19 Referencia Rápida**19.1 Comandos Esenciales**

```
# SERVIDOR
adb start-server      # Iniciar servidor
adb kill-server      # Detener servidor
adb version          # Versión

# DISPOSITIVOS
adb devices          # Listar dispositivos
adb devices -l       # Con detalles
adb -s SERIAL comando # Especificar dispositivo
adb -d comando       # Solo USB
adb -e comando       # Solo emulador

# CONEXIÓN
adb tcpip 5555        # Habilitar WiFi
adb connect IP:5555   # Conectar WiFi
adb disconnect        # Desconectar WiFi
adb usb              # Volver a USB

# INFORMACIÓN
```

```

adb shell getprop          # Propiedades
adb get-serialno           # Número de serie
adb get-state              # Estado

# APLICACIONES
adb install app.apk        # Instalar
adb install -r app.apk     # Reinstalar
adb uninstall PACKAGE     # Desinstalar
adb shell pm list packages # Listar apps
adb shell pm clear PACKAGE # Limpiar datos

# ARCHIVOS
adb push LOCAL REMOTE     # Copiar a dispositivo
adb pull REMOTE LOCAL     # Copiar de dispositivo
adb shell ls /sdcard      # Listar archivos

# SHELL
adb shell                  # Shell interactivo
adb shell COMANDO          # Comando único

# CAPTURA
adb exec-out screencap -p > screenshot.png
adb shell screenrecord /sdcard/video.mp4

# LOGS
adb logcat                 # Ver logs
adb logcat -c              # Limpiar logs
adb bugreport              # Reporte de bugs

# REINICIO
adb reboot                 # Reiniciar
adb reboot recovery        # Recovery
adb reboot bootloader      # Bootloader

```

19.2 Atajos de Teclado en Terminal

Atajo	Acción
Ctrl+C	Detener comando actual
Ctrl+D	Salir de shell adb
Ctrl+Z	Suspender comando
Ctrl+L	Limpiar pantalla
↑ / ↓	Historial de comandos

19.3 Variables de Entorno Útiles

```
# Fish
set -gx ANDROID_SERIAL ABC123456789
set -gx ANDROID_HOME /opt/android-sdk
set -gx ADB_TRACE all # Debug verbose

# Bash/Zsh
export ANDROID_SERIAL=ABC123456789
export ANDROID_HOME=/opt/android-sdk
export ADB_TRACE=all
```

19.4 Locations Importantes

/sdcard/	# Almacenamiento interno (equivale a /storage/emulated/0/)
/sdcard/DCIM/	# Fotos de cámara
/sdcard/Download/	# Descargas
/sdcard/Pictures/	# Imágenes
/sdcard/Music/	# Música
/sdcard/Movies/	# Videos
/data/data/	# Datos de apps (requiere root)
/system/app/	# Apps del sistema (requiere root)
/data/app/	# Apps de usuario (requiere root)

19.5 Intents Comunes

```
# Configuración
android.settings.SETTINGS
android.settings.WIFI_SETTINGS
android.settings.BLUETOOTH_SETTINGS
android.settings.APPLICATION_SETTINGS

# Acciones
android.intent.action.VIEW
android.intent.action.CALL
android.intent.action.SENDTO
android.intent.action.SEND
android.media.action.IMAGE_CAPTURE
```

20 Recursos Adicionales

20.1 Documentación Oficial

- **ADB Official:** <https://developer.android.com/studio/command-line/adb>
- **Platform Tools:** <https://developer.android.com/studio/releases/platform-tools>
- **Android Debug:** <https://developer.android.com/studio/debug>

20.2 Comunidad

- **XDA Developers:** <https://www.xda-developers.com/>
- **Stack Overflow:** <https://stackoverflow.com/questions/tagged/adb>
- **Android Developers:** <https://developer.android.com/>

20.3 Herramientas Complementarias

```
# En Arch Linux
sudo pacman -S android-tools android-udev scrcpy

# Otras herramientas útiles
yay -S apktool android-sdk-platform-tools
```

21 Mejores Prácticas

21.1 Seguridad

1. Depuración USB solo cuando sea necesario

- Desactivar cuando no uses ADB
- No conectar a computadoras desconocidas

2. Autorizar solo computadoras confiables

- Verificar huella digital RSA
- Revocar autorizaciones periódicamente

3. Conexión WiFi segura

- Usar solo en redes confiables
- Desactivar cuando no la uses
- No usar en redes públicas

21.2 Rendimiento

1. Cerrar servidor cuando no lo uses

```
adb kill-server
```

2. Usar comandos específicos

```
# Bien
adb -s ABC123 shell pm list packages -3

# Evitar
adb shell pm list packages | grep algo
```

3. Transferencias grandes por USB

- WiFi es más lento para archivos grandes
- USB 3.0 es significativamente más rápido

21.3 Desarrollo

1. Automatizar tareas repetitivas

- Crear scripts Fish/Bash
- Usar alias para comandos comunes

2. Mantener logs organizados

```
adb logcat > logs/app_$(date +%Y%m%d_%H%M%S).log
```

3. Backup regular

- Antes de pruebas destructivas
- Usar scripts automatizados

22 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1.  [Comandos De Informacion Windows](#)
2.  [Adb](#)
3.  [Limpieza Y Optimizacion De Pc](#)
4.  [Scrcpy](#)
5.  [Gestionar Versiones De Jdk En Kubuntu](#)
6.  [Instalar Tor Browser](#)
7.  [Crear Enlaces Duros O Hard Link En Linux](#)
8.  [Comandos Vim](#)
9.  [Guia De Git Y Github](#)
10.  [00 Primeros Pasos En Linux](#)
11.  [01 Introduccion Linux](#)
12.  [02 Distribuciones Linux](#)
13.  [03 Instalacion Linux](#)
14.  [04 Administracion Particiones Volúmenes](#)
15.  [Atajos De Teclado Y Comandos Para Usar Vim](#)
16.  [Instalando Specitify](#)
17.  [Gestiona Tus Dotfiles Con Gnu Stow](#)
18.  [Guia De Ranger](#)
19.  [Guia De Kitty Terminal](#)
20.  [Guia De Positron Ide](#)
21.  [Guia De Rsync](#)

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!